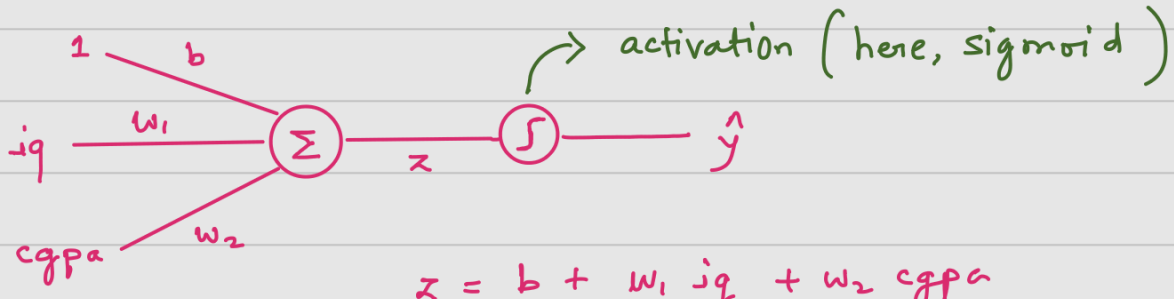
Perceptron Learning Algorithm

Initialize the weights w to small random values
Set the learning rate η
Repeat until convergence or a maximum number of iterations:
    For each training example (x, y):
        Calculate the predicted output ŷ = sign(w·x)
        Update the weights: w = w + η(y - ŷ)x

Perceptron with sigmoid

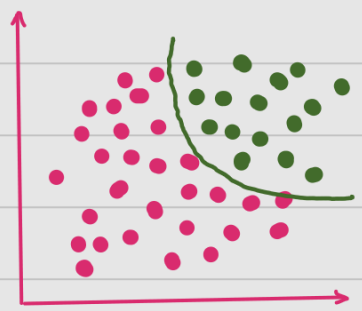


$$z = b + w_1\, iq + w_2\, cgpa$$

$$\hat{y} = \sigma(z)$$

Multi Layer Perceptron

Suppose we want to capture the following decision boundary.



The perceptron learning algorithm is a binary classification algorithm that was introduced by Frank Rosenblatt in 1957. It is a simple algorithm that can be used to learn a linear classifier, i.e., a function that can separate two classes of data points in a high-dimensional space.

The perceptron learning algorithm works as follows:

- Initialize the weights and bias to small random values.
- For each training example, compute the output of the perceptron using the current weights and bias.
- If the output is correct, do nothing. If the output is incorrect, adjust the weights and bias to move the decision boundary closer to the misclassified point.
- Repeat steps 2 and 3 for a fixed number of epochs or until the classification error is below a certain threshold.

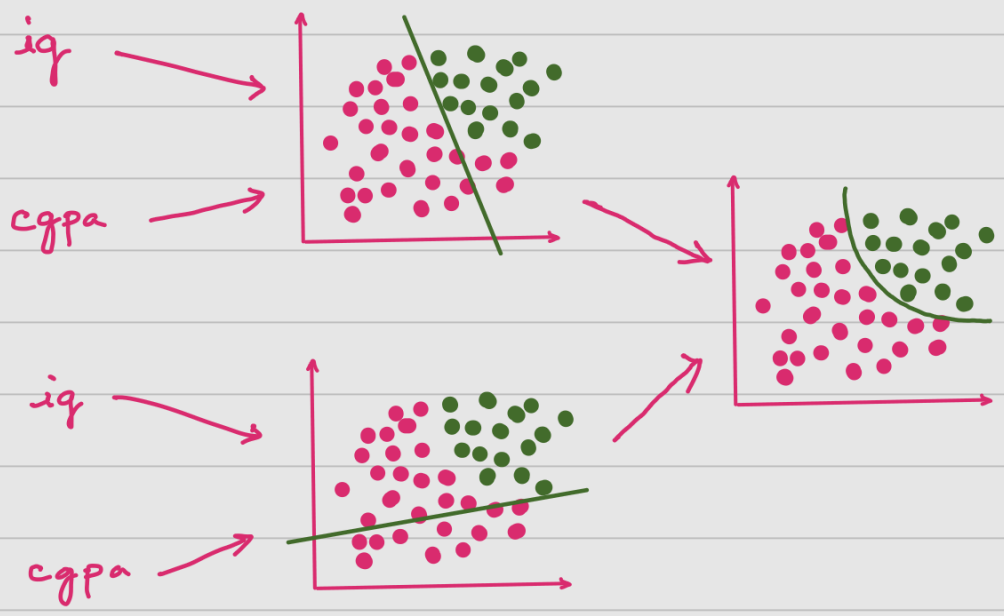
More formally, given a set of training examples {(x1, y1), (x2, y2), ..., (xn, yn)}, where xi is a d-dimensional feature vector and yi is its corresponding label (+1 or -1), the perceptron learning algorithm updates the weights and bias as follows:

```
if yi(w * xi + b) <= 0:
    w = w + yi * xi
    b = b + yi
```

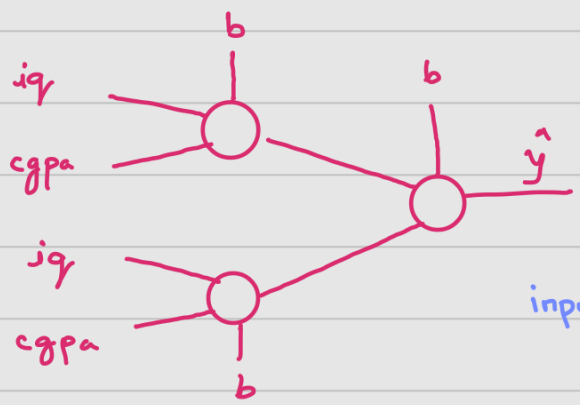
Here, w is a d-dimensional weight vector and b is a scalar bias term. The notation w * xi represents the dot product between the weight vector and the feature vector.

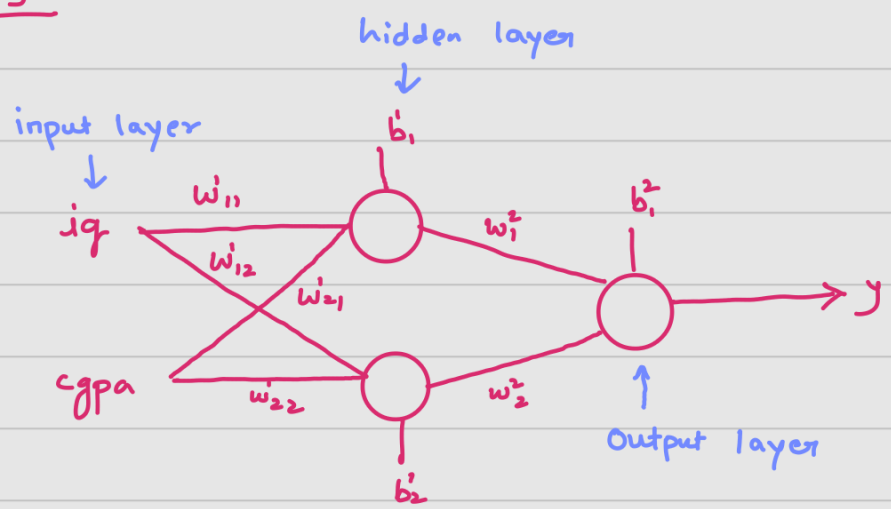
The perceptron learning algorithm is a simple and effective algorithm for linear classification problems. However, it has some limitations, such as the fact that it can only learn linear decision boundaries and may not converge if the data is not linearly separable.

We can utilize multiple perceptrons for that



Which is equivalent to the following two architectures.

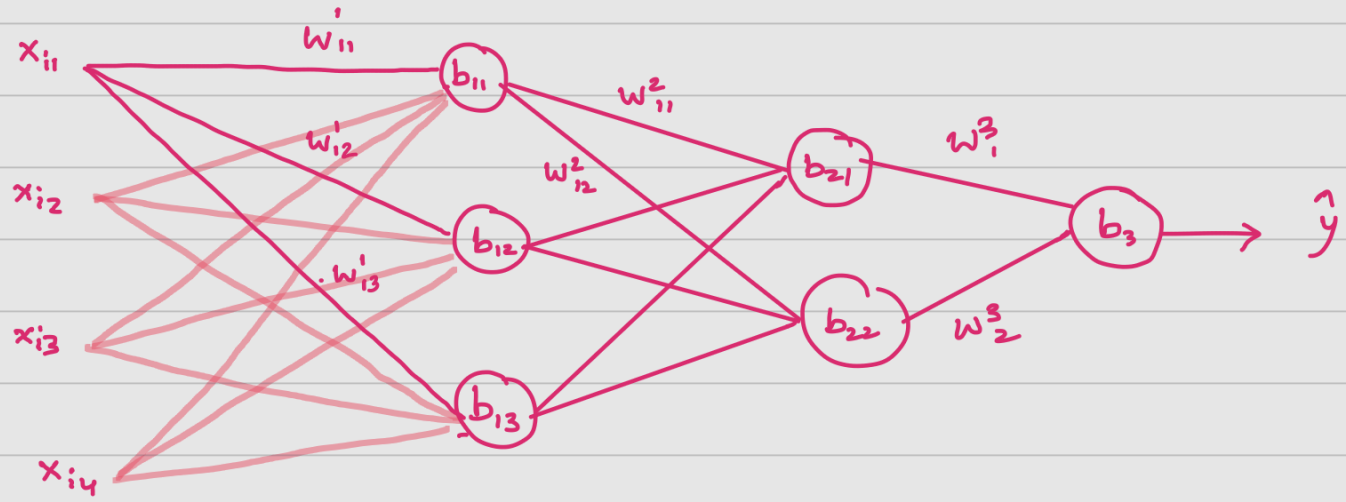




In order to capture more non linearity we can do the following

* add more features
* increase the depth of hidden layer
* add more hidden layers

Forward Propagation



Layer 1.

a row corresponding to weights going from x_{ij} to all perceptrons

$$\begin{bmatrix} w'_{11} & w'_{12} & w'_{13} \\ w'_{21} & w'_{22} & w'_{23} \\ w'_{31} & w'_{32} & w'_{33} \\ w'_{41} & w'_{42} & w'_{43} \end{bmatrix}^T \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} + \begin{bmatrix} b_{11} \\ b_{12} \\ b_{13} \end{bmatrix} = \begin{bmatrix} z_{11} \\ z_{12} \\ z_{13} \end{bmatrix}$$

col^m corresponding to weights from all x_{ij} to perceptron 1

$$\begin{bmatrix} w'_{11}x_1 + w'_{21}x_2 + w'_{31}x_3 + w'_{41}x_4 + b_{11} \\ \vdots \\ \vdots \end{bmatrix} = \begin{bmatrix} z_{11} \\ z_{12} \\ z_{13} \end{bmatrix}$$

and.

$$\begin{bmatrix} o_{11} \\ o_{12} \\ o_{13} \end{bmatrix} = \sigma\left\{\begin{bmatrix} z_{11} \\ z_{12} \\ z_{13} \end{bmatrix}\right\}$$

Layer 2.

$$\begin{bmatrix} w^2_{11} & w^2_{12} \\ w^2_{21} & w^2_{22} \\ w^2_{31} & w^2_{32} \end{bmatrix}^T \begin{bmatrix} o_{11} \\ o_{12} \\ o_{13} \end{bmatrix} + \begin{bmatrix} b_{21} \\ b_{22} \end{bmatrix} = \begin{bmatrix} z_{21} \\ z_{22} \end{bmatrix}$$

$$\begin{bmatrix} o_{21} \\ o_{22} \end{bmatrix} = \sigma \left\{ \begin{bmatrix} z_{21} \\ z_{22} \end{bmatrix} \right\}$$

Layer 3.

$$\begin{bmatrix} w^3_1 \\ w^3_2 \end{bmatrix}^T \begin{bmatrix} o_{21} \\ o_{22} \end{bmatrix} + \begin{bmatrix} b_3 \end{bmatrix} = \begin{bmatrix} z_3 \end{bmatrix}$$

$$\hat{y} = \sigma \{ z_3 \}$$

Backpropagation

For regression setting

Steps :-

initialize weights and biases

select a point (row)

predict → forward prop

choose a loss function (MSE)

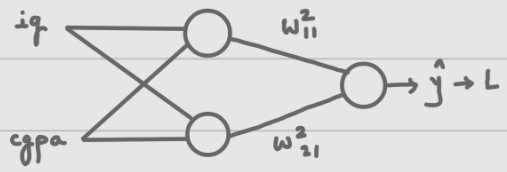
update weights and biases (Gradient descent)

$$w_{new} = w_{old} - \eta \frac{\partial L}{\partial w_{old}} \qquad b_{new} = b_{old} - \eta \frac{\partial L}{\partial b_{old}}$$

i.e. $$w_{11}^2 = w_{11}^2 - \eta \frac{\partial L}{\partial w_{11}^2} \qquad b_{11} = b_{11} - \eta \frac{\partial L}{\partial b_{11}}$$

$$\frac{\partial L}{\partial w_{11}^2} \quad \frac{\partial L}{\partial w_{21}^2} \quad \frac{\partial L}{\partial b_{21}} \qquad \frac{\partial L}{\partial w_{11}^1} \quad \frac{\partial L}{\partial w_{21}^1} \quad \frac{\partial L}{\partial b_{11}} \qquad \frac{\partial L}{\partial w_{12}^1} \quad \frac{\partial L}{\partial w_{22}^1} \quad \frac{\partial L}{\partial b_{12}}$$

$$\frac{\partial L}{\partial w_{11}^2} = \frac{\partial L}{\partial \hat{y}} \times \frac{\partial \hat{y}}{\partial w_{11}^2} \qquad \frac{\partial L}{\partial \hat{y}} = \frac{\partial}{\partial \hat{y}} (y-\hat{y})^2 = -2(y-\hat{y})$$



$$\frac{\partial \hat{y}}{\partial w_{11}^2} = \frac{\partial}{\partial w_{11}^2} \{ o_{11} w_{11}^2 + o_{12} w_{21}^2 + b_{21} \} = o_{11}$$

$$\therefore \frac{\partial L}{\partial w_{11}^2} = -2(y-\hat{y})\, o_{11}$$

Now, $$\frac{\partial L}{\partial w_{21}^2} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial w_{21}^2} = -2(y-\hat{y})\, o_{12}$$

Now, $$\frac{\partial L}{\partial b_{21}} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial b_{21}} = -2(y-\hat{y})$$

$$\frac{\partial L}{\partial w^1_{11}} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial o_{11}} \cdot \frac{\partial o_{11}}{\partial w^1_{11}} \qquad \frac{\partial L}{\partial w^1_{21}} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial o_{11}} \cdot \frac{\partial o_{11}}{\partial w^1_{21}}$$

and so on ...

For classification setting